context of a controller's action method and the view code will be presenting any errors back to the end user. And for many applications, that's the case.

But if we need to save a model object in a context where we want to make sure to handle all errors programmatically, we should use `save!`. This method raises a `RecordInvalid` exception if the object couldn't be saved:

```
begin
  order.save!
rescue RecordInvalid => error
  # validation failed
end
```

Deleting Rows

Active Record supports two styles of row deletion. First, it has two class-level methods, `delete` and `delete_all`, that operate at the database level. The `delete` method takes a single ID or an array of IDs and deletes the corresponding row(s) in the underlying table. `delete_all` deletes rows matching a given condition (or all rows if no condition is specified). The return values from both calls depend on the adapter but are typically the number of rows affected. An exception isn't thrown if the row doesn't exist prior to the call.

```
Order.delete(123)
User.delete([2,3,4,5])
Product.delete_all(["price > ?", @expensive_price])
```

The various `destroy` methods are the second form of row deletion provided by Active Record. These methods all work via Active Record model objects.

The `destroy` instance method deletes from the database the row corresponding to a particular model object. It then freezes the contents of that object, preventing future changes to the attributes.

```
order = Order.find_by(name: "Dave")
order.destroy
```